

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

86400

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

604800

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from scipy.signal import wiener

# 1. Generate a Synthetic Autonomous Driving Scene (Road & Lanes)
grid_size = 512
scene = np.zeros((grid_size, grid_size), dtype=np.uint8)

# Draw a perspective road layout
pts = np.array([[200, 200], [312, 200], [480, 512], [32, 512]], np.int32)
cv2.fillPoly(scene, [pts], 80)

# Draw bright lane markers (high spatial frequency targets)
cv2.line(scene, (256, 200), (256, 512), 255, 6, lineType=cv2.LINE_AA)
for y_pos in range(220, 512, 60):
    cv2.line(scene, (256, y_pos), (256, y_pos + 30), 0, 6)

# Draw a simulated pedestrian/obstacle box on the road
cv2.rectangle(scene, (236, 280), (276, 340), 180, -1)

# 2. FORWARD OPTICAL MODEL: Injecting Physical Camera Degradations
# Step A: Lens Barrel Distortion Model
distCoeff = np.zeros((4, 1), np.float32)
distCoeff[0, 0] = -0.4 # Negative value simulates wide-angle barrel distortion
```

```

camMatrix = np.eye(3, dtype=np.float32)
camMatrix[0, 0] = grid_size / 2.0
camMatrix[1, 1] = grid_size / 2.0
camMatrix[0, 2] = grid_size / 2.0
camMatrix[1, 2] = grid_size / 2.0

distorted_scene = cv2.undistort(scene, camMatrix, distCoeff)

# Step B: Vehicle-Shake Motion Blur Model (Linear Point Spread Function)
blur_size = 15
psf = np.zeros((blur_size, blur_size))
# Horizontal motion smear across the center row
psf[blur_size // 2, :] = 1.0
psf /= psf.sum()

# Convolve distorted image with the motion blur kernel
forward_degraded_stream = cv2.filter2D(distorted_scene, -1, psf)
# Add minimal sensor noise floor
noise = np.random.normal(0, 3, forward_degraded_stream.shape).astype(np.int16)
forward_degraded_stream = np.clip(forward_degraded_stream.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# 3. INVERSE PHYSICS MODEL: Restoring Lost Spatial Information
# Step A: Geometric Rectification (Inverting Lens Curvature)
# Generate optimal camera matrix to retain frame edges
newCamMatrix, _ = cv2.getOptimalNewCameraMatrix(camMatrix, distCoeff, (grid_size, grid_size), 1)
rectified_geometry = cv2.undistort(forward_degraded_stream, camMatrix, distCoeff, None, newCamMatrix)

# Step B: Wiener Deconvolution (Inverting Motion Blur)
# Use the known physical PSF model to de-blur the frame and recover sharp edges
# We utilize an adaptive noise-to-signal estimation ratio for restoration stability
restored_float = rectified_geometry.astype(float) / 255.0
inverse_restored = wiener(restored_float, mysize=(5, 5), noise=0.005)
inverse_restored = np.clip(inverse_restored * 255.0, 0, 255).astype(np.uint8)

# 4. Plot and Compare the Perception Pipeline States
plt.figure(figsize=(20, 10))

# Plot True Scene
plt.subplot(1, 4, 1)
plt.imshow(scene, cmap='gray')
plt.title("1. True Physical Scene\n(Perfect Spatial Geometry)", fontsize=11)
plt.axis('off')

# Plot Forward Degraded Output
plt.subplot(1, 4, 2)
plt.imshow(forward_degraded_stream, cmap='gray')
plt.title("2. Degraded Camera Capture\n(Barrel Warp + Vehicle Motion Blur)", fontsize=11)
plt.axis('off')

# Plot Geometric Rectification Intermediate State
plt.subplot(1, 4, 3)
plt.imshow(rectified_geometry, cmap='gray')
plt.title("3. Inverse Step A: Rectified Geometry\n(Straightened Lanes, Still Blurry)", fontsize=11)
plt.axis('off')

# Plot Complete Inverse Reconstruction
plt.subplot(1, 4, 4)
plt.imshow(inverse_restored, cmap='gray')
plt.title("4. Inverse Step B: Sharp Output\n(Motion Restored for Vehicle AI)", fontsize=11)
plt.axis('off')

plt.tight_layout()
plt.show()

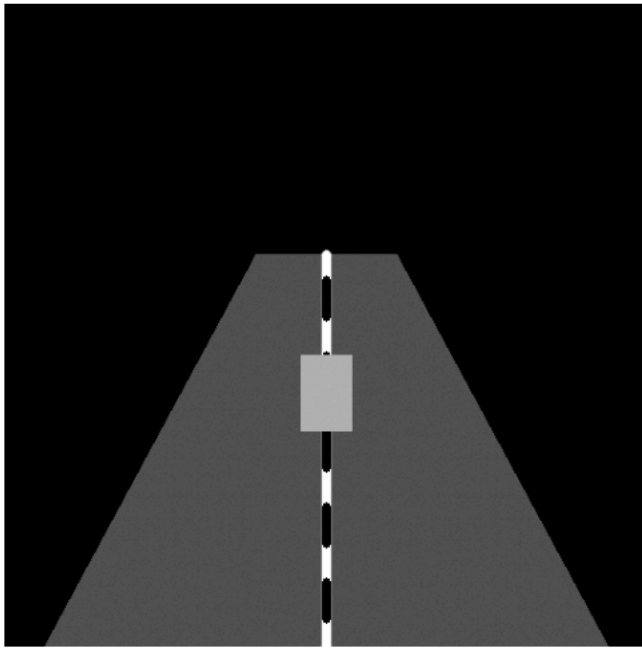
```

```

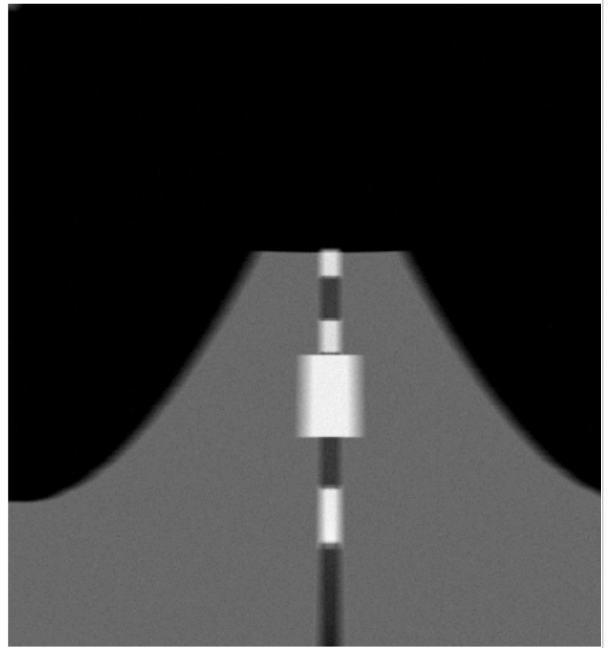
/usr/local/lib/python3.12/dist-packages/scipy/signal/_signaltools.py:1783: RuntimeWarning: divide by zero encountered in div
res *= (1 - noise / lVar)
/usr/local/lib/python3.12/dist-packages/scipy/signal/_signaltools.py:1783: RuntimeWarning: invalid value encountered in mult
res *= (1 - noise / lVar)

```

1. True Physical Scene
(Perfect Spatial Geometry)



2. Degraded Camera Capture
(Barrel Warp + Vehicle Motion Blur)



Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs](#) and [TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10 minute guide](#) or [US stock market data analysis demo](#).

Machine learning

With Colab you can import an image dataset, train an image classifier on it, and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

More Resources

Working with Notebooks in Colab

- [Overview of Colab](#)
- [Guide to Markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with Data

- [Loading data: Drive, Sheets, and Google Cloud Storage](#)
- [Charts: visualizing data](#)